

DatoScope

An Interactive Approach to Data Visualization and Machine Learning

MA5755 Final Project Report Department of Mathematics, IIT Madras

Aritra Dasgupta
(MA25M005)

Mehak Gupta
(MA25M016)

Tanmoy Ghosh
(MA25M026)

Supervised by Prof. Rakhi Singh

1. Introduction and Motivation

Modern data analysis is no longer limited to fitting a model and reporting a score. In practice, analysts must understand data quality, class imbalance, outliers, feature relationships, dimensionality and model stability before they can draw reliable conclusions. Our project, titled **DatoScope** is developed as an interactive system that combines exploratory data analysis, preprocessing, supervised learning, clustering and model comparison in a single Streamlit-based application.

The central research questions behind this project are:

1. How does the underlying structure of a dataset (linear or nonlinear) influence the performance of different regression models?
2. How do changes in noise levels influence the performance and stability of classification models?
3. How well do different clustering techniques identify meaningful patterns in datasets with varying geometrical structures and densities?
4. Can interactive visualization help improve the understanding of algorithm behavior and assist in selecting appropriate models for different types of data?

DatoScope attempts to bridge the gap between theory and applied understanding by providing an interactive environment in which the entire workflow can be seen, not just the final answer.

2. System Architecture and Project Structure

DatoScope utilizes a modular, multi-page Streamlit architecture.

2.1 Directory Layout

```

1 DatoScope/
2 |-- app.py                      # Main Streamlit launcher and sidebar navigation
3 |-- pages/
4 |   |-- 1_EDA.py                # Main landing page: overview + EDA
5 |   |-- 2_Supervised_Modeling.py # Regression and classification
6 |   |-- 3_Clustering.py         # Clustering workflows
7 |   +-- 4_Comparison.py         # Model comparison dashboard
8 |-- utils/
9 |   |-- app_state.py            # Session-state initialization and persistence
10 |   |-- data_input.py           # Sidebar controls, upload/generation workflow
11 |   |-- generators.py           # Synthetic dataset generation
12 |   |-- modeling.py             # Training, evaluation, export helpers
13 |   |-- preprocessing.py        # Cleaning, encoding, and preprocessing logic
14 |   |-- io.py                   # File and dataset I/O helpers
15 |   +-- ui.py                   # Shared styling and visual helpers
16 |-- scripts/
17 |   |-- 01_generate_data.py      # Offline synthetic data generation
18 |   |-- 02_clean_data.py         # Offline preprocessing
19 |   |-- 03_eda.py                # Offline exploratory analysis
20 |   +-- 04_visualization.py      # Offline static visualizations
21 |-- datasets/                   # Stored/generated datasets
22 |-- outputs/                     # Saved outputs, reports, and plots
23 |-- data_loader.py               # Standalone dataset loading utility
24 |-- train.py                     # Offline pipeline runner
25 |-- README.md                   # Project documentation
26 +-- requirements.txt             # Python dependencies

```

3. Data Input and Preprocessing

3.1 Data Input Modes

DatoScope supports three distinct data input workflows, all accessible from the persistent sidebar:

1. **Generate Dataset:** Select an ML task and dataset type, configure parameters (samples, noise, features, split ratio, etc.), and instantly generate data for the session.
2. **Upload Single File:** Upload a dataset (CSV, Excel, data, ZIP). The application automatically infers schemas and handles train/test splitting during the modeling phase.
3. **Upload Train/Test Files:** Upload separate train and test files to bypass automatic splitting, evaluating models directly on the provided test set.

3.2 Synthetic Dataset Catalogue

The `utils/generators.py` module provides synthetic datasets for **regression**, **classification**, and **clustering**. All generators inject missing values and outliers by default (approximately 5% and 3%, respectively), and the user can control sample size, noise, feature count, informative features, cluster count, target name, and random seed through the app.

Clustering Datasets: Gaussian Blobs, Two Moons, Concentric Circles, Spiral, Anisotropic Blobs, Variable Density Blobs

Regression Datasets: Linear, Nonlinear, Sinusoidal, High-Dimensional

Classification Datasets: Linearly Separable, Overlapping Classes, Imbalanced Classes

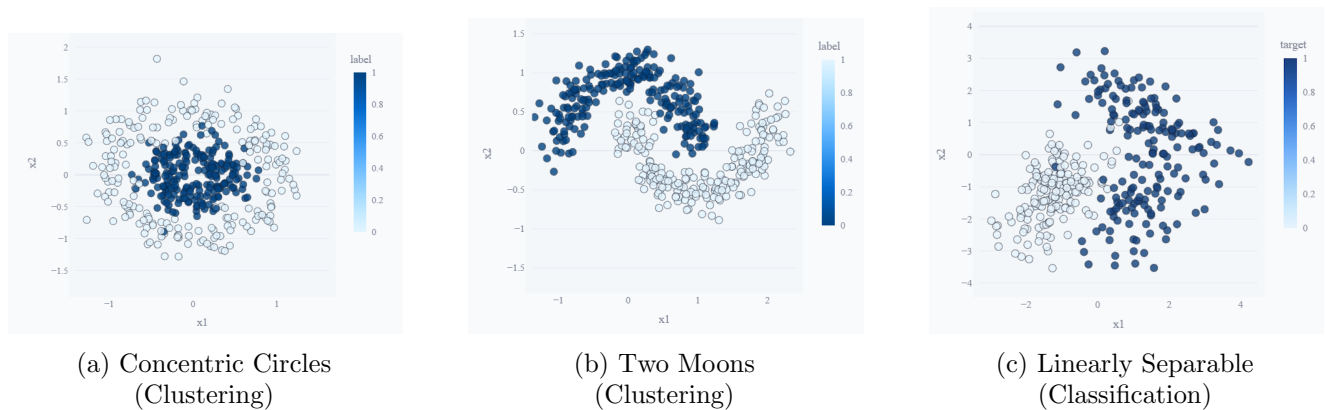


Figure 1: Synthetic Data Generation example with their specific Task category

For lower-dimensional generated datasets, the app can expand the feature space to the requested size by creating additional correlated features as noisy linear combinations of existing ones. This allows high-dimensional experiments while preserving interpretable underlying structure.

3.3 Preprocessing Pipeline

The preprocessing pipeline, implemented in `utils/preprocessing.py`, is applied separately to train and test datasets. It supports **missing-value handling** (mean, median, mode, or row deletion), **outlier removal** (IQR or Z-score based row filtering), **duplicate removal**, **numeric scaling** (Standard, Min-Max, or Robust), and **categorical encoding** (One-Hot or Label, including column-wise choices). Cleaned train and test data are stored separately in session state, enabling raw-versus-cleaned comparisons, cleaning logs, and CSV export. The interface also provides a missing-value summary and an estimate of how many rows the selected outlier method will remove before preprocessing is executed.

Cleaned Train Preview

Rows after cleaning

25,907

Columns after cleaning

21

Actions taken

5

Train cleaning log

Filled missing values with 'mean'

Z-Score: removed 3624 outlier row(s)

Scaled with StandardScaler

Label-encoded 2 categorical column(s)

One-hot encoded 1 categorical column(s) into 5 feature column(s)

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene	Xylene	AQI	AQI_Bucket_Gc
0	0	0	0.2377	0.1899	-1.0639	-0.4374	-0.5245	0.1975	-0.2082	1.7361	133.36	-0.6722	-0.9143	-1.4815	0.3134	
1	0	1	0.2377	0.1899	-1.0598	-0.5949	-0.5592	0.1975	-0.1828	1.3961	34.06	0.387	-0.1581	0.6107	0.3134	
2	0	2	0.2377	0.1899	0.279	-0.3702	0.1073	0.1975	8.1624	1.8935	30.7	1.285	1.3459	-0.2329	0.3134	
3	0	3	0.2377	0.1899	-1.0003	-0.4212	-0.4832	0.1975	0.188	0.7403	36.08	0.6028	0.4821	-0.9266	0.3134	
4	0	4	0.2377	0.1899	0.662	-0.2383	0.513	0.1975	10.5497	3.0225	39.31	1.3454	1.6895	0.0613	0.3134	
9	0	5	0.2377	0.1899	0.2932	-1.1332	-1.3878	0.1975	0.4667	-0.3932	4.55	-0.6722	-0.9171	-1.4815	0.3134	

Figure 2: Data Cleaning Log after Preprocessing, data from <https://www.kaggle.com/datasets/rohanrao/air-quality-data-in-india/>

4. Exploratory Data Analysis

The EDA page (`pages/1.EDA.py`) combines dataset overview, preprocessing outputs, and exploratory analysis for both training and test datasets.

- **Dataset Overview:** Shows dataset metadata, train/test preview tables, generated dataset preview plots, cleaned dataset previews, cleaning logs, download buttons for cleaned data, and missing value summaries.
- **Summary Statistics:** Displays standard descriptive statistics for numeric features together with skewness and kurtosis.
- **Feature Distributions:** User selected histograms arranged in a responsive subplot grid.
- **Box Plots:** Plotted for selected variables, with outlier percentages and IQR-based outlier bounds.
- **Q-Q Plots:** Quantile-quantile plots with reference lines to assess normality, skewness, and tail behavior.
- **Correlation Matrix:** A heatmap of the Pearson correlation matrix.
- **Scatter Plot with Trendline:** Scatter plots with OLS trendlines.
- **Feature Variance Ranking:** A variance based ranking of numeric features.

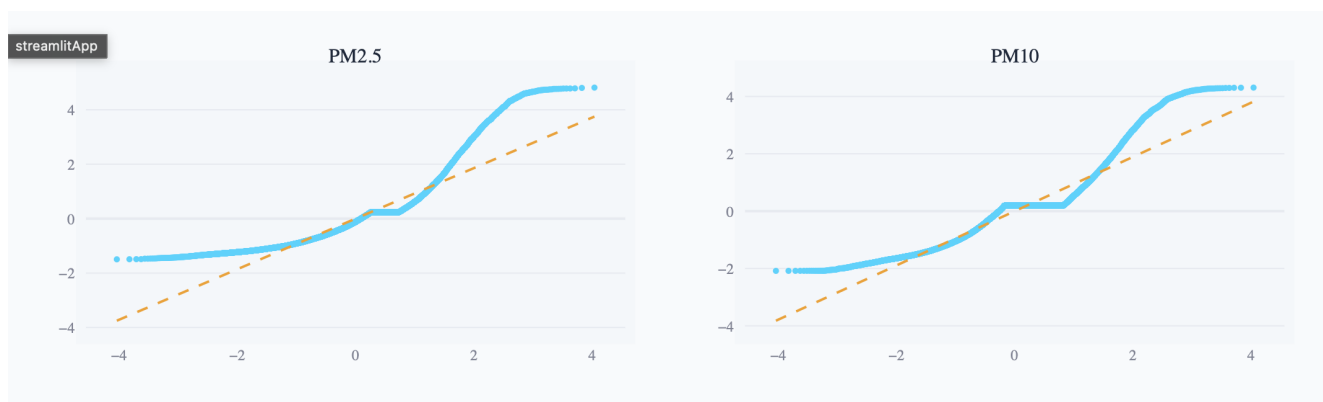


Figure 3: QQ Plots for PM2.5 and PM10 features for the data from <https://www.kaggle.com/datasets/rohanrao/air-quality-data-in-india/>

5. Supervised Modeling

The supervised modeling page ([pages/2.Supervised.Modeling.py](#)) provides a unified interface for regression and classification, auto-detecting the task type via the target’s data type and cardinality while allowing manual overrides. To prevent target selections from silently carrying over between tasks, regression and classification maintain separate session states.

5.1 Common Configuration

Before selecting task-specific models, users configure:

- **Target column:** Selectable from all columns in the training dataset.
- **Feature columns:** Multi-select from all non-target columns; non-numeric features are automatically excluded with a warning.
- **Test split percentage:** Slider from 10% to 40%, used when no external test file is uploaded.
- **Cross-validation folds:** Slider from 3 to 10 folds.
- **Random seed:** Integer input for reproducibility.

5.2 Regression Models

Three linear regression models are available:

Linear Regression. Ordinary least squares, serving as the baseline. No regularization parameter is required.

Ridge Regression. L_2 -regularized regression solving:

$$\hat{\beta}_{\text{Ridge}} = \arg \min_{\beta} \|y - X\beta\|_2^2 + \alpha \|\beta\|_2^2$$

The regularization strength α is selectable from the discrete set $\{0.001, 0.01, 0.1, 1, 10, 100\}$.

Lasso Regression. L_1 -regularized regression solving:

$$\hat{\beta}_{\text{Lasso}} = \arg \min_{\beta} \|y - X\beta\|_2^2 + \alpha \|\beta\|_1$$

The L_1 penalty promotes sparsity, effectively performing feature selection. Alpha is selectable from the same discrete set.

Each model is evaluated using a comprehensive set of metrics:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$\text{Overfit Gap} = R_{\text{train}}^2 - R_{\text{test}}^2$$

Cross-validated R^2 is computed using k -fold CV on the full training set, providing an estimate of generalization performance independent of the held-out split.

For each model, three visualizations are shown side-by-side:

1. **Actual vs. Predicted:** A scatter plot with a $y = x$ reference line; deviations indicate systematic prediction errors.
2. **Residual Histogram:** Distribution of residuals ($y - \hat{y}$); ideally centered near zero and approximately normal.
3. **Coefficient Plot:** Horizontal bar chart of feature coefficients, color-coded by sign, sorted by absolute magnitude.

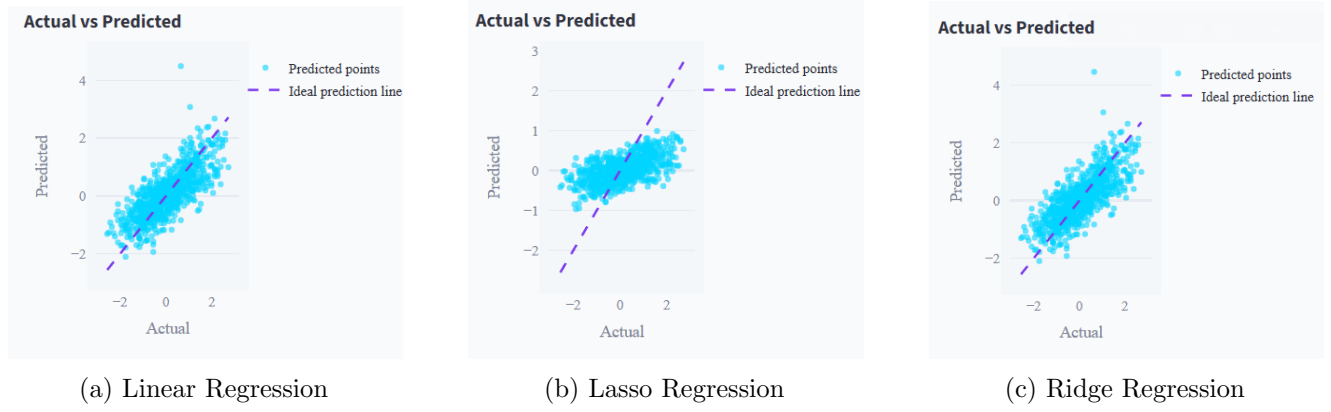


Figure 4: Regression Lines for all three models using the dataset from <https://www.kaggle.com/datasets/yasserh/wine-quality-dataset>

Model	R ²	CV R ²	Overfit Gap	RMSE	MAE	MSE
Linear Regression	0.5769	0.5977	0.0208	0.6467	0.4960	0.4182
Ridge (α=1)	0.5773	0.5977	0.0204	0.6463	0.4957	0.4177
Lasso (α=0.1)	0.2702	0.2706	0.0004	0.8492	0.6688	0.7212

Figure 5: Comparing all three regression models

5.3 Classification Models

Three classifiers are available:

Logistic Regression: A linear probabilistic classifier modeling:

$$P(y = 1 | x) = \sigma(x^\top \beta) = \frac{1}{1 + e^{-x^\top \beta}}$$

Random Forest: An ensemble of decision trees built by bagging, with the following user-configurable hyperparameters: estimators (50–500), max depth (1–30), min samples (split/leaf), max features (sqrt/log2), and bootstrap. Features an interactive, depth-adjustable visualization of individual trees.

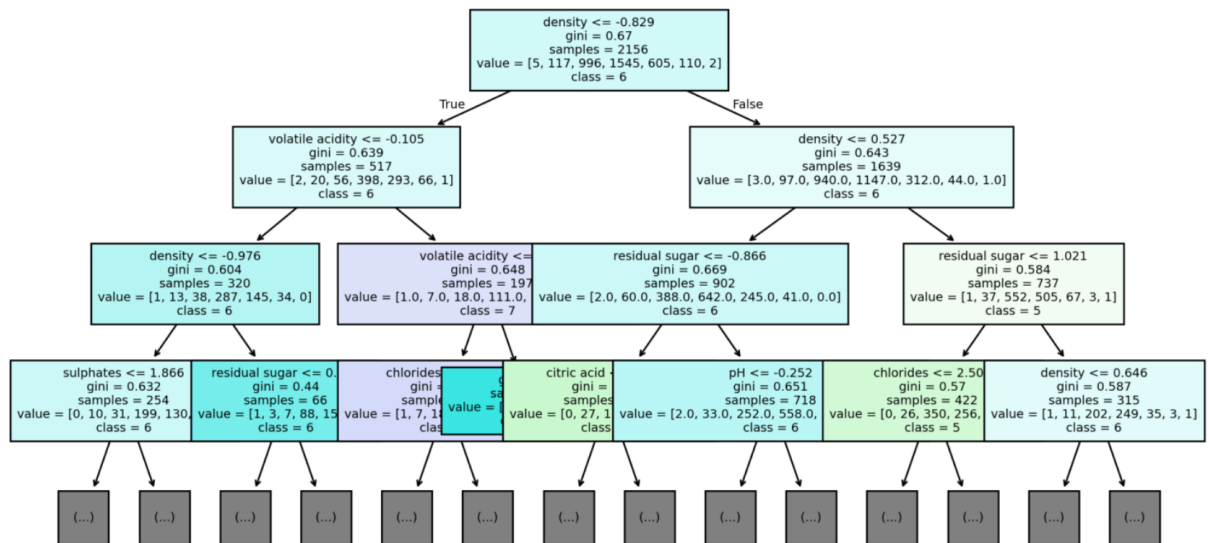


Figure 6: Tree structure for Random Forest for data from <https://www.kaggle.com/datasets/yasserh/wine-quality-dataset>

K-Nearest Neighbors (KNN): A non-parametric instance-based classifier with user-configurable k (1–25 neighbors).

Evaluation Metrics: Reports cross-validated Accuracy, Precision, Recall, and F1, alongside an interactive confusion matrix and per-class classification report for each model.

	precision	recall	f1-score	support
False	0.9855	0.9645	0.9749	141
True	0.9929	0.9972	0.995	705
accuracy	0.9917	0.9917	0.9917	0.9917
macro avg	0.9892	0.9809	0.985	846
weighted avg	0.9917	0.9917	0.9917	846

(a) Logistic Regression

	precision	recall	f1-score	support
False	1	0.9716	0.9856	141
True	0.9944	1	0.9972	705
accuracy	0.9953	0.9953	0.9953	0.9953
macro avg	0.9972	0.9858	0.9914	846
weighted avg	0.9953	0.9953	0.9952	846

(b) Random Forest

	precision	recall	f1-score	support
False	0.9718	0.9787	0.9753	141
True	0.9957	0.9943	0.995	705
accuracy	0.9917	0.9917	0.9917	0.9917
macro avg	0.9838	0.9865	0.9851	846
weighted avg	0.9918	0.9917	0.9917	846

(c) kNN

Figure 7: Evaluation Metrics for all three Classification Models, data from <https://www.kaggle.com/datasets/yasserh/wine-quality-dataset>

6. Clustering

The clustering page ([pages/3_Clustering.py](#)) runs unsupervised algorithms on the active training dataset. The page is accessible only when the selected ML task is set to “Clustering” or “Auto”, preventing accidental use on labeled supervised datasets.

6.1 Algorithms

K-Means: The standard Lloyd’s algorithm partitioning data into k spherical clusters by minimizing within-cluster sum of squares:

$$\arg \min_C \sum_{j=1}^k \sum_{x \in C_j} \|x - \mu_j\|^2$$

Configurable k value (2–10). Automatically generates an **Elbow Curve (inertia vs. k)** to help users visually determine the optimal cluster count.

DBSCAN: Density-Based Spatial Clustering of Applications with Noise identifies arbitrary-shaped clusters by growing regions of high density. Users configure:

- ϵ (eps): The neighborhood radius (0.01–10.0).
- `min_samples`: Minimum points required to form a dense region (2–20).

Points not reachable from any core point are labeled as noise (cluster -1) and displayed distinctly. DBSCAN is the only algorithm that can return a variable and non-predetermined number of clusters.

Hierarchical (Agglomerative) Clustering: Builds a cluster tree bottom-up by iteratively merging the closest pairs of clusters. Users specify:

- `n_clusters`: The number of flat clusters to cut from the dendrogram (2–10).
- Linkage criterion: Ward, complete, average, or single.

A **Dendrogram** (truncated to 30 leaves for legibility) is rendered using SciPy’s `linkage` and `dendrogram` functions, allowing visual inspection of the cluster hierarchy.



(a) kMeans



(b) DBScan



(c) Hierarchical Clustering

Figure 8: All three Clustering algorithms applied to the wine dataset <https://www.kaggle.com/datasets/yasserh/wine-quality-dataset>

6.2 Evaluation Metrics

When the dataset contains a `label` column (as all generated clustering datasets do), ground-truth-dependent metrics (Fowlkes-Mallows Score and Rand Index) are automatically computed in addition to the intrinsic metrics.

Table 1: Clustering evaluation metrics used in DatoScope

Metric	Range	Interpretation
Silhouette Score	$[-1, 1]$ $\uparrow$	Similarity to own cluster vs. neighboring clusters.
Davies-Bouldin	$[0, \infty)$ $\downarrow$	Avg. ratio of within-cluster scatter to between-cluster separation.
Calinski-Harabasz	$[0, \infty)$ $\uparrow$	Ratio of between-cluster to within-cluster dispersion.
Fowlkes-Mallows	$[0, 1]$ $\uparrow$	Geometric mean of pairwise precision and recall (supervised).
Rand Index	$[0, 1]$ $\uparrow$	Fraction of consistent pairwise label assignments (supervised).

Dimensionality Reduction: Applies PCA to project high-dimensional data into 2D cluster scatter plots when the selected feature set has more than two dimensions. Axes display the percentage of explained variance to indicate preserved information.

7. Model Comparison Dashboard

The comparison page (`pages/4.Comparison.py`) aggregates the results of all trained regression, classification, and clustering models and explains both *which* model is selected as best and *why*.

Regression Comparison: Regression models are ranked using a composite **Generalization Score** built from normalized versions of **test R^2** , **cross-validated R^2** , **inverse-normalized RMSE**, **inverse-normalized MAE**, and **inverse-normalized Overfit Gap**. The winning model is selected from the highest overall generalization score, combining strong predictive performance, stability and lower overfitting risk through different metrics.

Classification Comparison: Classification models are ranked by **Macro F1**, which gives equal importance to all classes and is therefore more reliable than plain accuracy under class imbalance. **Accuracy**, **weighted F1**, and **cross-validated Accuracy** are shown as supporting metrics for interpretation.

Clustering Comparison: Clustering models are compared using a **Metric Wins** approach. For each available metric, the best algorithm receives one win: **highest Silhouette**, **lowest Davies-Bouldin**, **highest Calinski-Harabasz**, **highest Fowlkes-Mallows**, and **highest Rand Index**. The algorithm with the largest number of wins is selected as best. Ties are broken in order by **Silhouette**, then **Rand Index**, and then **Fowlkes-Mallows Score**.

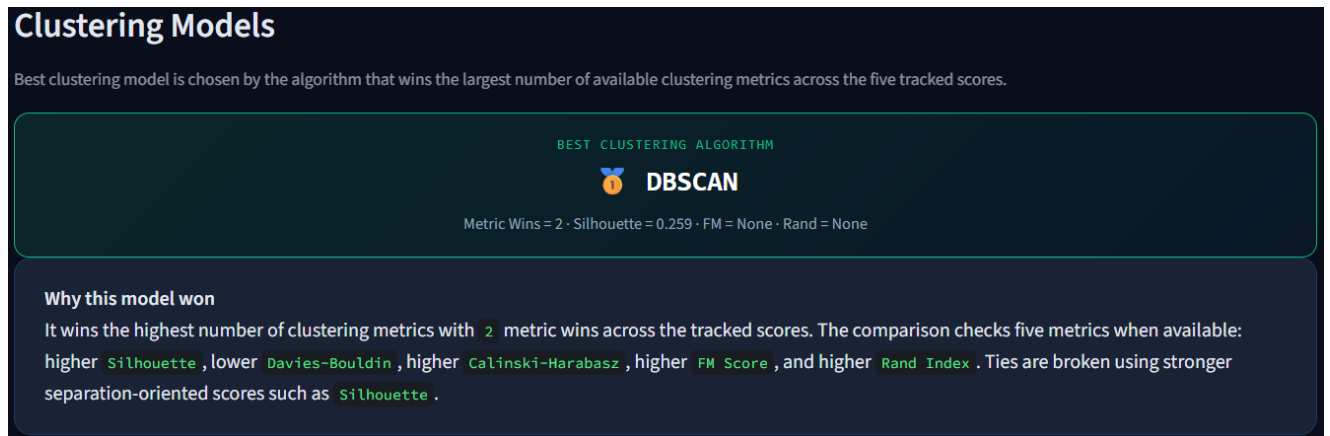


Figure 9: DBScan wins in Clustering for the wine dataset from <https://www.kaggle.com/datasets/yasserh/wine-quality-dataset>

Algorithm	Clusters found	Silhouette $\uparrow$	Davies-Bouldin $\downarrow$	Calinski-Harabasz $\uparrow$
K-Means	3	0.2161	1.6119	1066.2325
DBSCAN	3	0.2590	1.0848	478.8963
Hierarchical	3	0.1954	1.5689	902.2670

Figure 10: DBScan has better scores in two out of three metrics in Clustering for the wine dataset from <https://www.kaggle.com/datasets/yasserh/wine-quality-dataset>

8. Research Questions and Findings

This section summarizes observations from the interactive tool across five core experiment configurations.

• Effect of Dataset Structure on Regression:

	Model	R^2	CV R^2	RMSE	MAE	Overfit Gap	Generalization Score
0	Linear Regression	0.974	0.9701	0.1613	0.1241	0.0039	0.88
1	Ridge ($\alpha=1$)	0.9737	0.9699	0.1621	0.1263	0.0038	0.2789
2	Lasso ($\alpha=0.01$)	0.9734	0.9699	0.1631	0.1259	0.0035	0.1418

Figure 11: Comparison of regression models on generated linear data. Since the underlying signal is approximately linear, Linear Regression performs best in both fit quality and generalization while Ridge and Lasso offer no substantial advantage from regularization in this setting.

	Model	R^2	CV R^2	RMSE	MAE	Overfit Gap	Generalization Score
0	Linear Regression	0.732	0.844	0.5177	0.3489	0.1119	0.28
1	Ridge ($\alpha=10$)	0.7354	0.8407	0.5144	0.3445	0.1053	0.7682
2	Lasso ($\alpha=0.1$)	0.7347	0.8336	0.5151	0.3343	0.0989	0.6202

Figure 12: Comparison of regression models on generated nonlinear data. Because the true relationship is nonlinear, all three models experience a drop in predictive performance. Ridge performs better overall, indicating that regularization provides modest robustness.

• Effect of Noise on Classification:

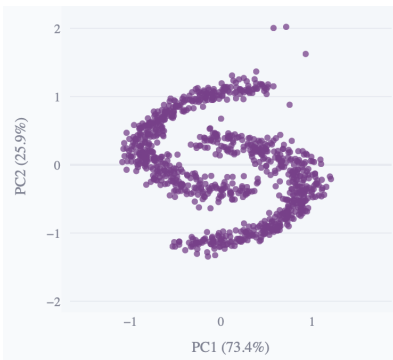
Model	Accuracy	Precision	Recall	F1	Macro F1	CV Accuracy
Logistic Regression	0.8587	0.8602	0.8587	0.8585	0.8585	0.8793
Random Forest (250)	0.9348	0.9356	0.9348	0.9348	0.9348	0.9396
KNN (5)	0.9457	0.9459	0.9457	0.9456	0.9456	0.9606

Figure 13: Classification performance on the generated dataset with noise level 0.05. At low noise, all models perform well, with KNN achieving the highest Macro F1 and CV Accuracy, indicating that clear local class structure supports both accuracy and stability. Although the noise is low, the scores are slightly weaker than in the higher-noise case, suggesting that model performance depends not only on noise but also on the class distribution and train-test composition.

Model	Accuracy	Precision	Recall	F1	⋮	Macro F1	CV Accuracy
Logistic Regression	0.9655	0.9658	0.9655	0.9655		0.9654	0.9567
Random Forest (250)	0.9885	0.9887	0.9885	0.9885		0.9884	0.9566
KNN (5)	0.9885	0.9887	0.9885	0.9885		0.9884	0.9617

Figure 14: Classification performance on the generated dataset with noise level 1.00. Despite the higher noise, Random Forest and KNN retain very strong Macro F1 values, suggesting that these models remain robust when class boundaries are still sufficiently separable under the generated distribution. Here the higher-noise dataset produced better scores because the class distribution was more balanced, showing that class balance can offset or even outweigh the negative effect of added noise.

• Clustering Under Different Geometries:



(a) kMeans

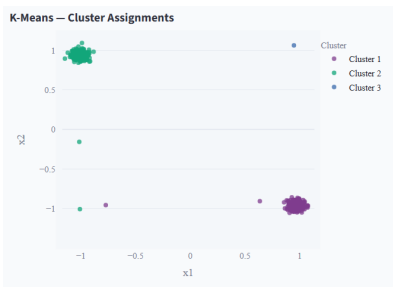


(b) DBScan

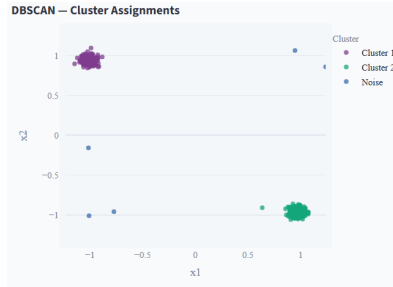


(c) Hierarchical Clustering (Ward)

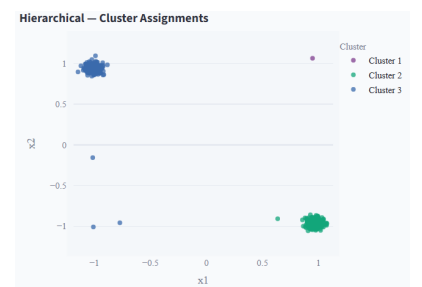
Figure 15: K-Means & Hierarchical fail on two-moons because they rely on distance to centroids or pairwise distances, implicitly assuming convex, spherical clusters, unable to capture curved, crescent-shaped boundaries. DBScan succeeds because it groups points by local density and neighborhood connectivity, following the shape of the data regardless of geometry.



(a) kMeans



(b) DBScan



(c) Hierarchical Clustering (Ward)

Figure 16: All three work on Gaussian blobs because blobs are compact, convex, and well-separated perfectly matching the centroid/distance assumptions that K-Means and Hierarchical make, and dense enough for DBScan's neighborhood criterion to work cleanly.

- **Interactive Visualization as a Learning Tool:** Interactive visualization improves understanding of algorithm behavior by allowing users to inspect data structure, outliers, class balance, correlations, residuals, and cluster separation before and after modeling. In DatoScope, visual tools such as scatter plots, box plots, Q-Q plots, confusion matrices, dendrograms, and comparison dashboards make model strengths and weaknesses easier to interpret than raw numerical output alone. This helps users connect dataset characteristics with model performance and therefore supports more informed selection of appropriate models for different types of data.

9. Implementation Details

Session State Management: Persistent application state is managed through `st.session_state` via `utils/app_state.py`. Key stored objects include `train_df`, `test_df`, `clean_train_df`, `clean_test_df`, `data_meta`, `reg_results`, `cls_results`, `cluster_results`, uploaded-file signatures, and separate target selections for regression and classification.

Navigation: The official launcher is `app.py`, which defines the sidebar navigation for **EDA**, **Supervised Modeling**, **Clustering**, and **Comparison**. The EDA page (`pages/1_EDA.py`) acts as the main landing page and also includes the dataset overview and preprocessing outputs.

UI and Styling: Shared visual styling is centralized in `utils/ui.py`, including the Plotly theme, dataset preview blocks, metadata cards, and reusable interface styling, supporting light(default) and dark themes.

Model Export: Trained supervised models can be downloaded as serialized pickle files. Each exported object stores the fitted model, selected feature list, target column, and task type, allowing reuse without retraining.

Offline Pipeline: The project also includes `train.py` and the `scripts/` pipeline for non-interactive experimentation. This workflow supports synthetic data generation, preprocessing, EDA, and static visualization output.

10. Code

The code is available at <https://github.com/tanmoyghosh704-lang/DatoScope>. The app is hosted at <https://datoscopemaster.streamlit.app>. It is open-source and can be freely used and modified by anyone as they want.

11. Limitations

There are many limitations of the current version of application, but most importantly:

- **Limited algorithm support:** Currently, only a few algorithms are implemented. Future work could introduce generative algorithms, ensemble methods such as XGBoost, and neural networks.
- **Dataset generation constraints:** The generation of synthetic datasets is restricted to a maximum of 5000 data points with at most 50 features.
- **Visualization dimensionality:** The application is currently limited to rendering only 2D plots for data visualization.

12. Future Extensions

- **Dimensionality reduction:** Interactive PCA, t-SNE, and UMAP implementation.
- **Feature importance:** SHAP value integration for model interpretability beyond coefficient plots.
- **Automated hyperparameter search:** Grid search or Bayesian optimization through the UI.
- **Dataset versioning:** The ability to save and reload named dataset and model configurations.
- **Time series support:** Rolling statistics, autocorrelation plots, and time-aware train/test splits.

13. Conclusion

DatoScope successfully demonstrates how an interactive platform can unify data preprocessing, exploratory analysis, supervised learning, clustering and model comparison within a single coherent workflow. In this project, it is shown that model performance is strongly shaped by dataset structure, noise, class balance, geometry and hyperparameter choice and that these effects become far more interpretable when supported by visualization rather than raw metrics alone.

By combining statistical diagnostics with task-specific comparison logic, the app DatoScope helps users move beyond simply fitting models toward understanding "why" a model succeeds or fails. As both a learning tool and a practical experimentation environment, it highlights the value of transparent, visual and reproducible data science workflows.